1. Introduction

The transition from analogue to digital in healthcare is a critical step towards improving operational efficiency and patient safety. The *Crownfield Eye Hospital NHS Foundation Trust*, within the fictional scenario of this project, faced administrative bottlenecks due to a reliance on paper-based records, Word documents, and unstructured Excel spreadsheets. This report documents the complete development cycle of a relational database prototype designed by *Flytech Ltd* to replace these inefficient processes. The scope of the project encompasses conceptual modelling, physical implementation in MySQL, and data manipulation using advanced SQL.

During development, certain assumptions were made to guide the modelling: it was assumed that clinicians can attend multiple clinics, but each appointment is allocated to one specific clinic. Furthermore, it was assumed that a patient could receive multiple diagnoses and several prescriptions during a single medical appointment.

The initial construction of the schema was a collaborative effort, but my individual contribution focused heavily on the physical implementation (DDL), the formulation of complex analytical queries (DQL), and the application of security mechanisms and transactions (TCL). This document will detail the rationale behind the design choices, the normalisation strategy, and the critical evaluation of the final system.

2. Analysis and design (Task A)

The starting point of the project was the analysis of an unnormalised ophthalmology dataset provided in the assessment brief. The analysis of this spreadsheet revealed severe database anomalies:

2.1 Dataset Analysis and Anomalies

- **Insertion Anomalies:** It was not possible to register a new clinician or clinic without a patient being scheduled, as there were no independent tables.
- **Update Anomalies:** Changing a clinic's address or a patient's postcode would require scanning and updating multiple rows, increasing the risk of inconsistency.

- **Deletion Anomalies:** If a patient's appointment was cancelled and the row deleted from the spreadsheet, the patient's data would be permanently lost from the system.

Appendix 1 as supplied

Patient ID	Patient (Non-atomic)	Postcode	Clinician	Specialty	Clinic	Date	Time	Diagnosis	Medicines	Follow up	Notes
OP001 (Redundancy)	Amina Yusuf	NE1 4AB	Dr Maya Shah	Glaucoma	City Road	2026-10-05	09:00	Ocular hypertension	Latanoprost | Timolol (Repeating group.)	6 weeks | High (Composite value.)	Review IOP
OP002	Daniel Reed	N16 7QP	Dr Leo Grant	Retina	City Road	2026-10-05	09:30	Diabetic macular oedema	Aflibercept	4 weeks | Urgent	OCT before review
OP001 (Redundancy)	Amina Yusuf	NE1 4AB	Dr Maya Shah	Glaucoma	Bedford	2026-11-16	10:15	Ocular hypertension	Latanoprost	12 weeks | Routine	Visual field test
OP003	Sofia Bennett	E8 2LD	Dr Maya Shah	Glaucoma	City Road	2026-10-06	11:00	Glaucoma suspect	None (Inconsistent representation)	26 weeks | Routine	Monitor
OP004	Michael Okoro	SE6 3RT	Dr Priya Nair	Cornea	Croydon	2026-10-07	14:00	Keratoconus	Sodium chloride drops	8 weeks | Routine	Corneal topography

2.2 Normalisation Process (UNF to 3NF): To resolve this, the normalisation process was strictly applied. In the **First Normal Form (1NF)**, we ensured that all attributes were atomic, eliminating repeating groups by separating the full name into first_name and last_name, splitting follow-up details into weeks and priority, and ensuring each prescribed medicine occupied a separate row under the composite key (appointment_id, medicine).

Table in 1NF

appt_id	patient_id	first_name	last_name	postcode	clinician	specialty	clinic	date	time	diagnosis	medicine	weeks	priority	notes
1	OP001	Amina	Yusuf	NE1 4AB	Dr Maya Shah	Glaucoma	City Road	2026-10-05	09:00	Ocular hypertension	Latanoprost	6	High	Review IOP
1	OP001	Amina	Yusuf	NE1 4AB	Dr Maya Shah	Glaucoma	City Road	2026-10-05	09:00	Ocular hypertension	Timolol	6	High	Review IOP
2	OP002	Daniel	Reed	N16 7QP	Dr Leo Grant	Retina	City Road	2026-10-05	09:30	Diabetic macular oedema	Aflibercept	4	Urgent	OCT before review
3	OP001	Amina	Yusuf	NE1 4AB	Dr Maya Shah	Glaucoma	Bedford	2026-11-16	10:15	Ocular hypertension	Latanoprost	12	Routine	Visual field test
4	OP003	Sofia	Bennett	E8 2LD	Dr Maya Shah	Glaucoma	City Road	2026-10-06	11:00	Glaucoma suspect	(none)	26	Routine	Monitor
5	OP004	Michael	Okoro	SE6 3RT	Dr Priya Nair	Cornea	Croydon	2026-10-07	14:00	Keratoconus	Sodium chloride drops	8	Routine	Corneal topography

In the **Second Normal Form (2NF)**, we removed partial dependencies from the composite key by decomposing the table into Appointment and AppointmentMedicine, as appointment attributes depend solely on appointment_id rather than on the medicine.

Appointment (PK: appointment_id)

appt_id	patient_id	first_name	last_name	postcode	clinician	specialty	clinic	date	time	diagnosis	weeks	priority	notes
1	OP001	Amina	Yusuf	NE1 4AB	Dr Maya Shah	Glaucoma	City Road	2026-10-05	09:00	Ocular hypertension	6	High	Review IOP
2	OP002	Daniel	Reed	N16 7QP	Dr Leo Grant	Retina	City Road	2026-10-05	09:30	Diabetic macular oedema	4	Urgent	OCT before review
3	OP001	Amina	Yusuf	NE1 4AB	Dr Maya Shah	Glaucoma	Bedford	2026-11-16	10:15	Ocular hypertension	12	Routine	Visual field test
4	OP003	Sofia	Bennett	E8 2LD	Dr Maya Shah	Glaucoma	City Road	2026-10-06	11:00	Glaucoma suspect	26	Routine	Monitor
5	OP004	Michael	Okoro	SE6 3RT	Dr Priya Nair	Cornea	Croydon	2026-10-07	14:00	Keratoconus	8	Routine	Corneal topography

AppointmentMedicine (PK: appointment_id + medicine)

appointment_id	medicine
1	Latanoprost
1	Timolol
2	Aflibercept
3	Latanoprost
5	Sodium chloride drops

Appointment 4 has no medicine, so it has no row here.

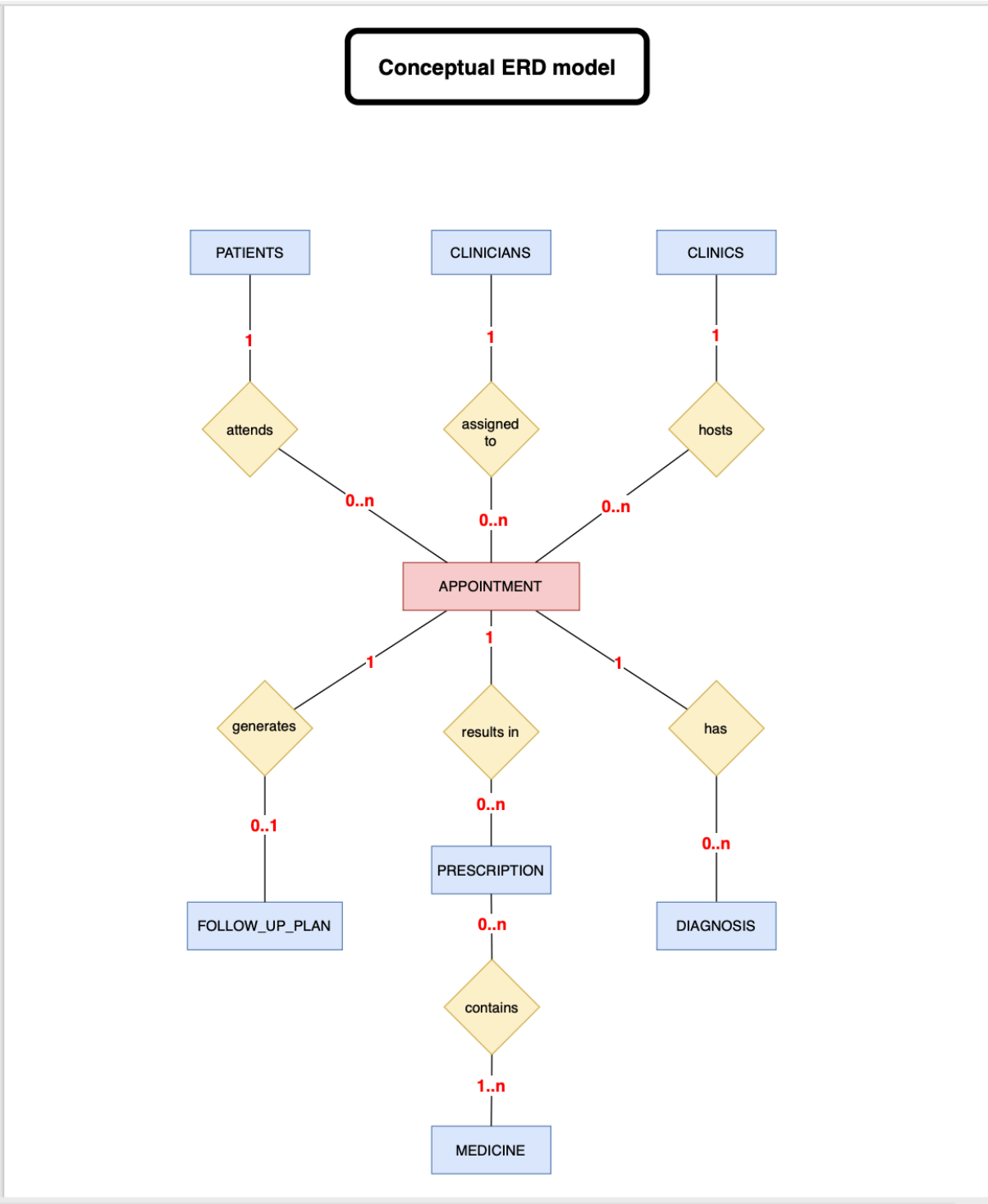
Finally, in the **Third Normal Form (3NF)**, we eliminated transitive dependencies and applied the hospital's business rules: patient details (such as postcode), clinicians, and clinics were moved into independent tables (patients, clinicians, and clinics), whilst AppointmentMedicine and clinical records were decomposed into prescriptions, medicines, prescription_medicines, diagnoses, and follow_up_plans.

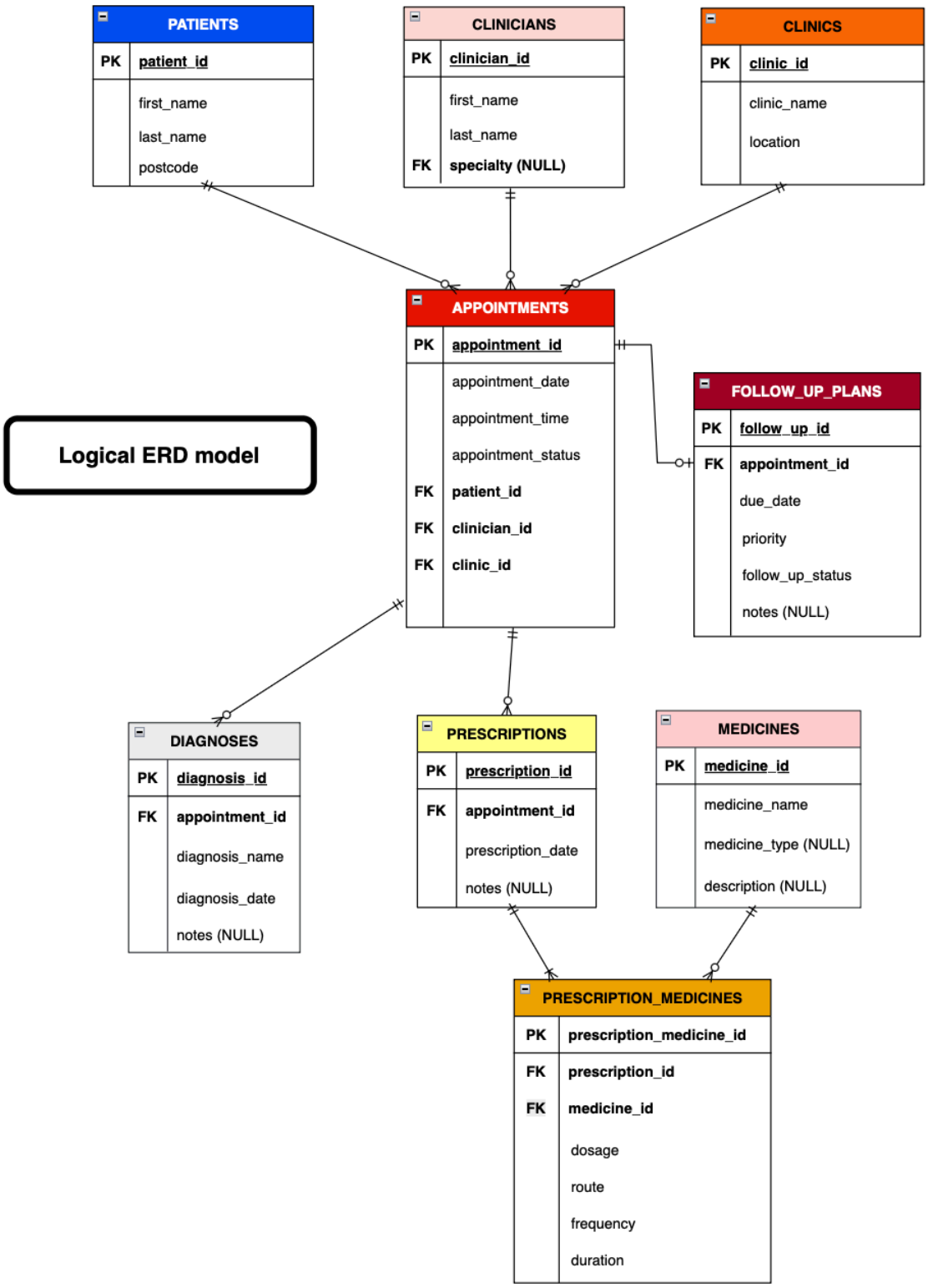
Table	Primary key	Foreign keys	Other attributes and constraints
patients	patient_id	–	first_name, last_name, postcode (all NOT NULL)
clinicians	clinician_id	–	first_name, last_name (NOT NULL); specialty (NULL allowed)
clinics	clinic_id	–	clinic_name (NOT NULL); location (NULL allowed)
appointments	appointment_id	patient_id → patients; clinician_id → clinicians; clinic_id → clinics	appointment_date, appointment_time (NOT NULL); appointment_status (DEFAULT 'Booked'); notes (NULL allowed)
diagnoses	diagnosis_id	appointment_id → appointments	diagnosis_name, diagnosis_date (NOT NULL); notes (NULL allowed)
prescriptions	prescription_id	appointment_id → appointments	prescription_date (NOT NULL); notes (NULL allowed)
medicines	medicine_id	–	medicine_name (NOT NULL); medicine_type, description (NULL allowed)
prescription_medicines	prescription_medicine_id	prescription_id → prescriptions; medicine_id → medicines	dosage, route, frequency, duration (NOT NULL); UNIQUE (prescription_id, medicine_id)
follow_up_plans	follow_up_id	appointment_id → appointments	appointment_id UNIQUE; due_date (NOT NULL); priority (DEFAULT 'Routine', CHECK Routine, High or Urgent); follow_up_status (DEFAULT 'Pending'); notes (NULL allowed)

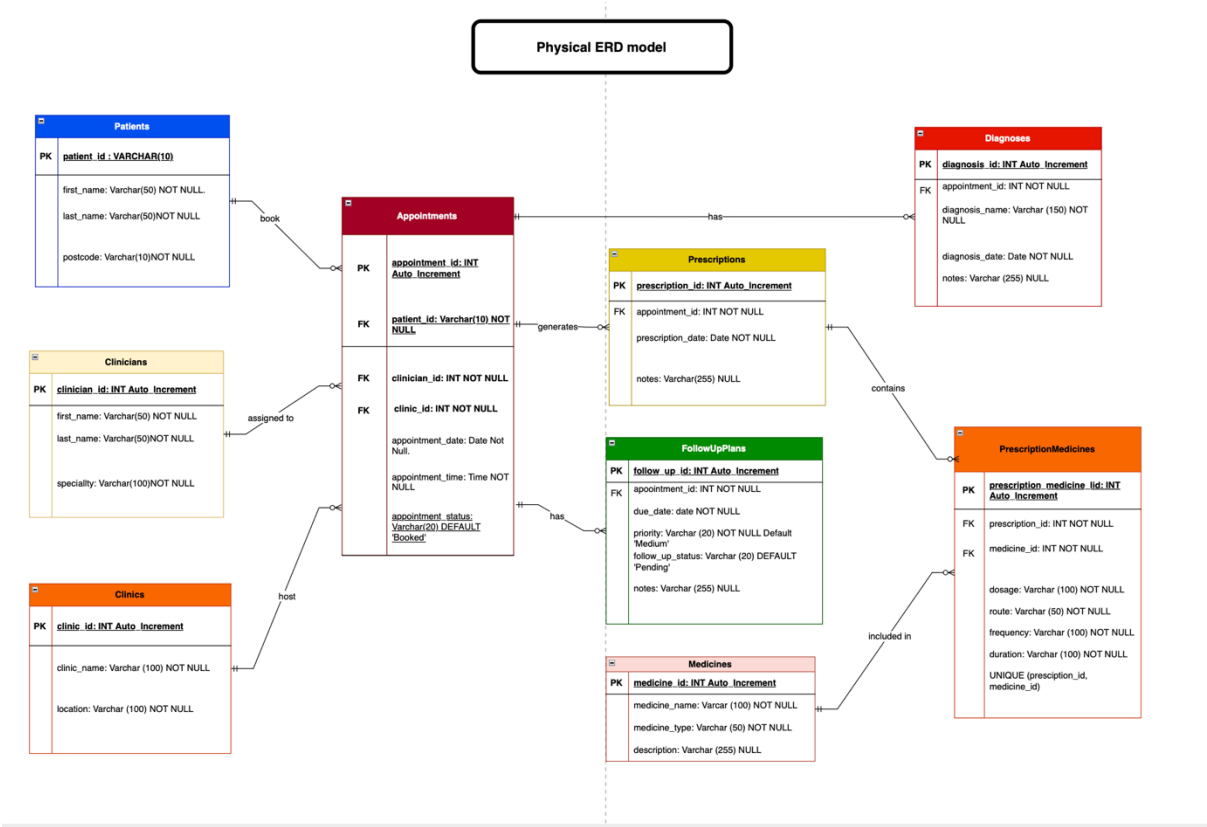
2.3 Entity-Relationship Modelling (ERD)

The conceptual Entity-Relationship Diagram was constructed using Chen notation with relationship diamonds to visually outline the core clinical business rules, whilst the subsequent Logical and Physical diagrams were developed using Crow's Foot notation. The appointments entity acts as the central transactional table. The cardinality was strictly defined: one patient (1) can have many (M) appointments, but each appointment belongs to only one (1) patient. Similarly, one-to-many (1:M) relationships were established between appointments and diagnoses. No many-to-

many (M:N) relationships were left in the logical model, ensuring that there were no orphaned tables and that structural integrity was maintained. Following the conceptual model, the logical and physical diagrams were developed to explicitly define data types, primary keys (PK), and foreign key (FK) constraints.

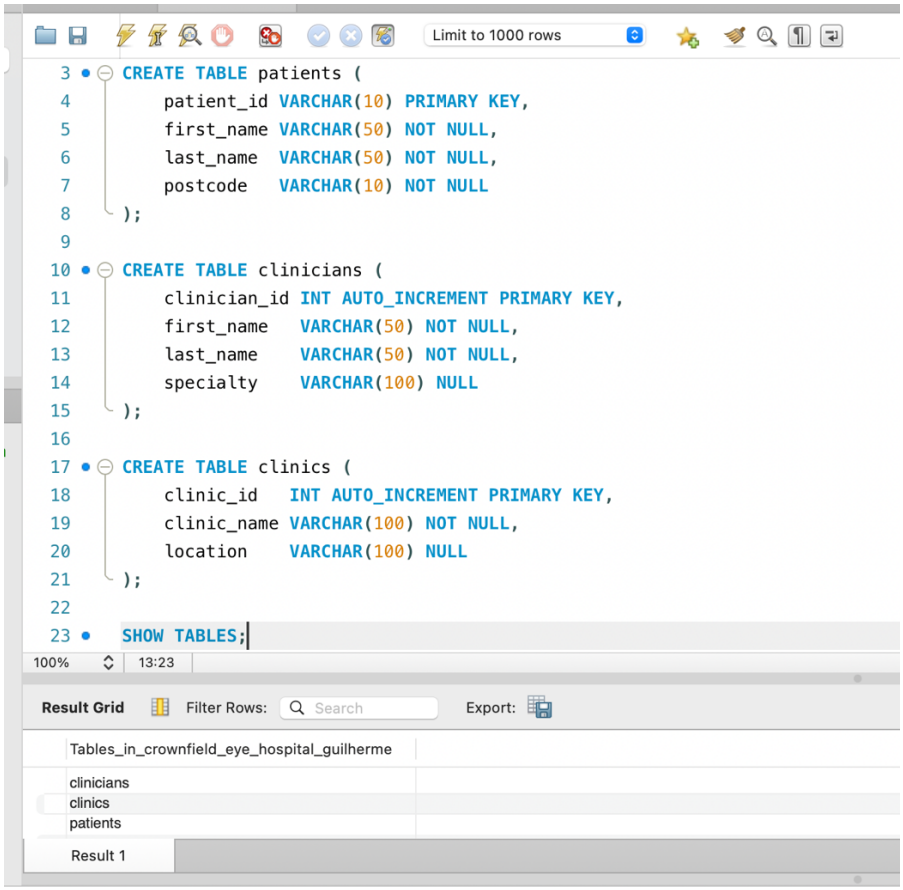



PATIENTS
PK patient_id
first_name
last_name
postcode
CLINICIANS
PK clinician_id
first_name
last_name
FK specialty (NULL)
CLINICS
PK clinic_id
clinic_name
location
APPOINTMENTS
PK appointment_id
appointment_date
appointment_time
appointment_status
FK patient_id
FK clinician_id
FK clinic_id
FOLLOW_UP_PLANS
PK follow_up_id
FK appointment_id
due_date
priority
follow_up_status
notes (NULL)
Logical ERD model
DIAGNOSES
PK diagnosis_id
FK appointment_id
diagnosis_name
diagnosis_date
notes (NULL)
PRESCRIPTIONS
PK prescription_id
FK appointment_id
prescription_date
notes (NULL)
MEDICINES
PK medicine_id
medicine_name
medicine_type (NULL)
description (NULL)
PRESCRIPTION_MEDICINES
PK prescription_medicine_id
FK prescription_id
FK medicine_id
dosage
route
frequency
duration



3. Implementation and Testing (Task B)

3.1 DDL Schema and Constraints: The transition from the logical model to the physical implementation was carried out in the MySQL environment. The DDL script (01_schema.sql) was written with consistent naming conventions (using snake_case). Data types were chosen carefully: VARCHAR for variable text fields such as names to optimise space, native DATE and TIME for appointments, and INT for clinic and clinician IDs. Integrity constraints were the pillar of this stage. The PRIMARY KEY constraint was declared in all tables to ensure the uniqueness of records. FOREIGN KEY constraints were fundamental to link the appointments table to patients, clinicians, and clinics. Furthermore, I utilised NOT NULL constraints in critical fields (such as appointment dates and mandatory foreign keys) to prevent the entry of incomplete operational records.



```sql
CREATE TABLE patients (
    patient_id VARCHAR(10) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name  VARCHAR(50) NOT NULL,
    postcode   VARCHAR(10) NOT NULL
);

CREATE TABLE clinicians (
    clinician_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name   VARCHAR(50) NOT NULL,
    last_name    VARCHAR(50) NOT NULL,
    specialty    VARCHAR(100) NULL
);

CREATE TABLE clinics (
    clinic_id   INT AUTO_INCREMENT PRIMARY KEY,
    clinic_name VARCHAR(100) NOT NULL,
    location    VARCHAR(100) NULL
);

SHOW TABLES;
```

Limit to 1000 rows

100% 13:23

Result Grid Filter Rows: Search Export:

Tables_in_crownfield_eye_hospital_guilherme
clinicians
clinics
patients

Result 1

```sql
USE crownfield_eye_hospital_guilherme;

CREATE TABLE appointments (
    appointment_id     INT AUTO_INCREMENT PRIMARY KEY,
    patient_id         VARCHAR(10) NOT NULL,
    clinician_id       INT NOT NULL,
    clinic_id          INT NOT NULL,
    appointment_date   DATE NOT NULL,
    appointment_time   TIME NOT NULL,
    appointment_status VARCHAR(20) NOT NULL DEFAULT 'Booked',
    notes              VARCHAR(255) NULL,
    FOREIGN KEY (patient_id)   REFERENCES patients(patient_id),
    FOREIGN KEY (clinician_id) REFERENCES clinicians(clinician_id),
    FOREIGN KEY (clinic_id)    REFERENCES clinics(clinic_id)
);

SHOW TABLES;
```

100% 13:17

Result Grid Filter Rows: Search Export:

Tables_in_crownfield_eye_hospital_guilherme
appointments
clinicians
clinics
patients

3.2 Data Population: For the testing phase, I populated the relational schema with a total of more than 10 coherent synthetic records across the entity tables using INSERT INTO commands. The insertion hierarchy was strictly respected (independent tables first) so as not to violate foreign key constraints.

```sql
USE crownfield_eye_hospital_guilherme;

INSERT INTO patients (patient_id, first_name, last_name, postcode) VALUES
('P001', 'John', 'Smith', 'L1 4DQ'),
('P002', 'Emma', 'Watson', 'L3 2BX');

INSERT INTO clinicians (first_name, last_name, specialty) VALUES
('Sarah', 'Jenkins', 'Ophthalmology'),
('David', 'Miller', 'Optometry');

INSERT INTO clinics (clinic_name, location) VALUES
('Glaucoma Clinic', 'Building A – Room 102'),
('Cornea Clinic', 'Building B – Room 205');

INSERT INTO appointments (patient_id, clinician_id, clinic_id, appointment_date, appointment_time, appointment_status, notes) VALUES
('P001', 1, 1, '2026-11-10', '09:00:00', 'Booked', 'Routine pressure check'),
('P002', 2, 2, '2026-11-10', '10:30:00', 'Booked', 'Post-op follow-up');

SELECT * FROM appointments;
```

100% 28:19

Result Grid Filter Rows: Search Edit: Export/Import:

appointment...	patient_id	clinician_id	clinic_id	appointment_d...	appointment_ti...	appointment_stat...	notes
1	P001	1	1	2026-11-10	09:00:00	Booked	Routine pressure check
2	P002	2	2	2026-11-10	10:30:00	Booked	Post-op follow-up
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

appointments 1

```sql
USE crownfield_eye_hospital_guilherme;

INSERT INTO diagnoses (appointment_id, diagnosis_name, diagnosis_date, notes) VALUES
(1, 'Ocular Hypertension', '2026-11-10', 'Monitor intraocular pressure closely.'),
(2, 'Dry Eye Syndrome', '2026-11-10', 'Prescribed artificial tears and lifestyle adjustments.');

INSERT INTO medicines (medicine_name, medicine_type, description) VALUES
('Latanoprost', 'Eye Drops', 'Prostaglandin analog used to treat elevated intraocular pressure.'),
('Hypromellose', 'Lubricant Eye Drops', 'Artificial tears for dry eye relief.');

INSERT INTO prescriptions (appointment_id, medicine_id, dosage, duration, instructions) VALUES
(1, 1, '1 drop', '30 days', 'Apply 1 drop in the affected eye once daily at night.'),
(2, 2, '1–2 drops', '14 days', 'Apply as needed for dryness up to 4 times a day.');

SELECT * FROM prescriptions;
```

100% 29:15

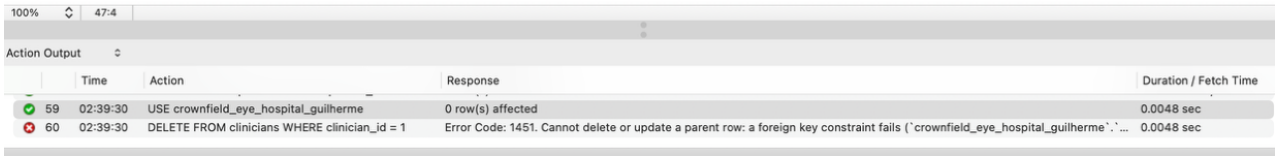
Result Grid Filter Rows: Search Edit: Export/Import:

prescription...	appointment...	medicine_id	dosage	duration	instructions
1	1	1	1 drop	30 days	Apply 1 drop in the affected eye once daily at night.
2	2	2	1-2 drops	14 days	Apply as needed for dryness up to 4 times a day.
NULL	NULL	NULL	NULL	NULL	NULL

prescriptions 1

3.3 Referential Integrity: The robustness of the database was tested by executing a controlled DELETE operation on a clinician who had active appointments. As expected, MySQL rejected the deletion due to the foreign key constraint, proving that referential integrity is actively protecting the database against the accidental deletion of vital data.

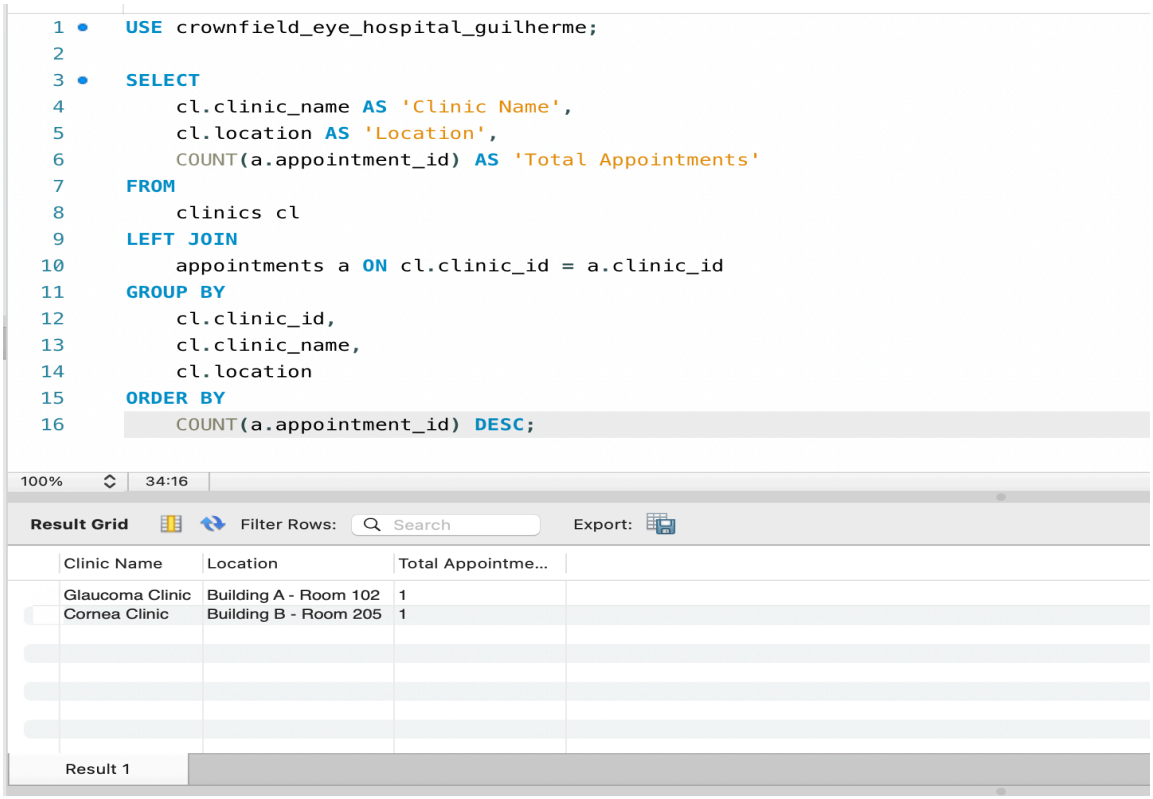
```
1 • USE crownfield_eye_hospital_guilherme;
2
3
4 • DELETE FROM clinicians WHERE clinician_id = 1;
```



4. Individual SQL Queries (Task C)

With the database structured and populated, I focused on demonstrating the capacity for data manipulation and retrieval through six individual SQL queries, designed to answer specific business questions for the hospital:

4.1 Aggregate Query (GROUP BY and ORDER BY): This query was developed to evaluate clinical attendance volume across hospital facilities. I utilised the COUNT () function alongside GROUP BY and a LEFT JOIN to calculate the total appointments per clinic, including facilities with low bookings. The ORDER BY COUNT (a.appointment_id) DESC clause ranks facilities hierarchically by patient demand, assisting hospital administration in capacity planning and resource allocation.



```sql
USE crownfield_eye_hospital_guilherme;

SELECT
    cl.clinic_name AS 'Clinic Name',
    cl.location AS 'Location',
    COUNT(a.appointment_id) AS 'Total Appointments'
FROM
    clinics cl
LEFT JOIN
    appointments a ON cl.clinic_id = a.clinic_id
GROUP BY
    cl.clinic_id,
    cl.clinic_name,
    cl.location
ORDER BY
    COUNT(a.appointment_id) DESC;
```

100% 34:16

Result Grid Filter Rows: Search Export:

Clinic Name	Location	Total Appointme...
Glaucoma Clinic	Building A - Room 102	1
Cornea Clinic	Building B - Room 205	1

Result 1

4.2 Multiple Join Schedule Report (INNER JOIN): To generate an operational daily schedule for reception staff, I utilised INNER JOIN to link the appointments, patients, and clinicians tables. This query resolves numerical foreign keys into human-readable data, outputting patient and clinician full names, appointment date, time, and current booking status in a consolidated view.

```sql
USE crownfield_eye_hospital_guilherme;

SELECT
    a.appointment_id AS 'Appt ID',
    CONCAT(p.first_name, ' ', p.last_name) AS 'Patient Name',
    CONCAT(c.first_name, ' ', c.last_name) AS 'Clinician Name',
    a.appointment_date AS 'Date',
    a.appointment_time AS 'Time',
    a.appointment_status AS 'Status'
FROM
    appointments a
INNER JOIN
    patients p ON a.patient_id = p.patient_id
INNER JOIN
    clinicians c ON a.clinician_id = c.clinician_id
ORDER BY
    a.appointment_date ASC;
```

100% 28:17

Result Grid Filter Rows: Search Export:

Appt ID	Patient Name	Clinician Name	Date	Time	Status
1	John Smith	Sarah Jenkins	2026-11-10	09:00:00	Booked
2	Emma Watson	David Miller	2026-11-10	10:30:00	Booked

4.3 Non-Pharmacological Audit Report (LEFT JOIN): I implemented an outer join hierarchy using LEFT JOIN between appointments, prescriptions, and medicines. This query ensures complete audit visibility by retaining all patient consultation records within the report, preserving appointment details even when an encounter concludes without a pharmacological prescription.

```sql
USE crownfield_eye_hospital_guilherme;

SELECT
    p.patient_id AS 'Patient ID',
    CONCAT(p.first_name, ' ', p.last_name) AS 'Patient Name',
    a.appointment_date AS 'Date',
    d.diagnosis_name AS 'Diagnosis',
    m.medicine_name AS 'Prescribed Medicine',
    pr.dosage AS 'Dosage'
FROM
    patients p
INNER JOIN
    appointments a ON p.patient_id = a.patient_id
INNER JOIN
    diagnoses d ON a.appointment_id = d.appointment_id
LEFT JOIN
    prescriptions pr ON a.appointment_id = pr.appointment_id
LEFT JOIN
    medicines m ON pr.medicine_id = m.medicine_id;
```

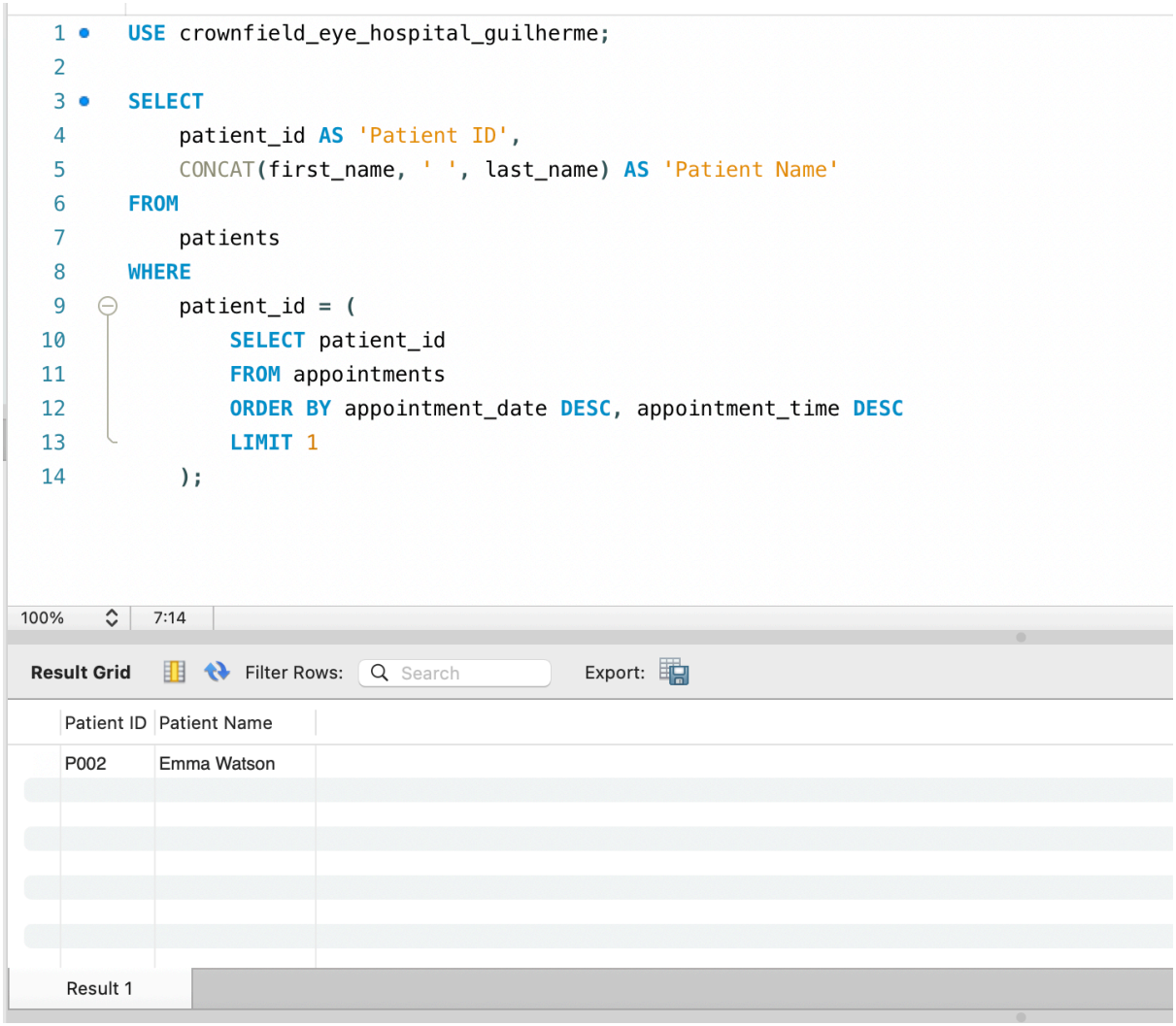
100% 51:19

Result Grid Filter Rows: Search Export:

Patient ID	Patient Name	Date	Diagnosis	Prescribed Medici...	Dosage
P001	John Smith	2026-11-10	Ocular Hypertension	Latanoprost	1 drop
P002	Emma Watson	2026-11-10	Dry Eye Syndrome	Hypromellose	1-2 drops

Result 1

4.4 Dynamic Subquery for Latest Appointment: I utilised a scalar subquery in the WHERE clause to dynamically retrieve the patient associated with the most recent clinic booking. Rather than hardcoding identifiers, the inner query isolates the target record using ORDER BY appointment_date DESC, appointment_time DESC LIMIT 1, enabling the outer query to return the corresponding patient details.



```sql
USE crownfield_eye_hospital_guilherme;

SELECT
    patient_id AS 'Patient ID',
    CONCAT(first_name, ' ', last_name) AS 'Patient Name'
FROM
    patients
WHERE
    patient_id = (
        SELECT patient_id
        FROM appointments
        ORDER BY appointment_date DESC, appointment_time DESC
        LIMIT 1
    );
```

100% 7:14

Result Grid Filter Rows: Search Export:

Patient ID	Patient Name
P002	Emma Watson

Result 1

4.5 Patient History Stored Procedure: I created a *Stored Procedure* named GetPatientHistory that accepts the patient ID as an input parameter (IN p_id VARCHAR (10)). When called with CALL GetPatientHistory('P001'), it immediately returns all past appointments for that patient. This reduces code repetition on the application side and improves performance.

```
1  USE crownfield_eye_hospital_guilherme;
2
3  DELIMITER //
4  CREATE PROCEDURE GetPatientHistory(IN p_id VARCHAR(10))
5  BEGIN
6      SELECT
7          a.appointment_date AS 'Date',
8          a.appointment_status AS 'Status',
9          d.diagnosis_name AS 'Diagnosis'
10     FROM
11         appointments a
12     LEFT JOIN
13         diagnoses d ON a.appointment_id = d.appointment_id
14     WHERE
15         a.patient_id = p_id
16     ORDER BY
17         a.appointment_date DESC;
18 END //
19 DELIMITER ;
20
21 CALL GetPatientHistory('P001');
```

100% 1:2

Result Grid Filter Rows: Search Export:

Date	Status	Diagnosis
2026-11-10	Booked	Ocular Hypertension

Result 1

4.6 Business Rule Enforcement Trigger: To enforce a hospital rule, I developed a BEFORE INSERT Trigger on the appointments table. Before a row is saved, the script verifies if the provided date is earlier than the current date (CURDATE ()). If it is, the Trigger aborts the operation and issues an error message indicating that "Cannot book an appointment in the past". This delegates validation directly to the database.

```sql
USE crownfield_eye_hospital_guilherme;

DELIMITER //
CREATE TRIGGER trg_prevent_past_appointments
BEFORE INSERT ON appointments
FOR EACH ROW
BEGIN
    IF NEW.appointment_date < CURDATE() THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Cannot book an appointment in the past.';
    END IF;
END //
DELIMITER ;

INSERT INTO appointments (patient_id, clinician_id, clinic_id, appointment_date, appointment_time, appointment_status)
VALUES ('P002', 1, 1, '2020-01-01', '09:00:00', 'Booked');
```

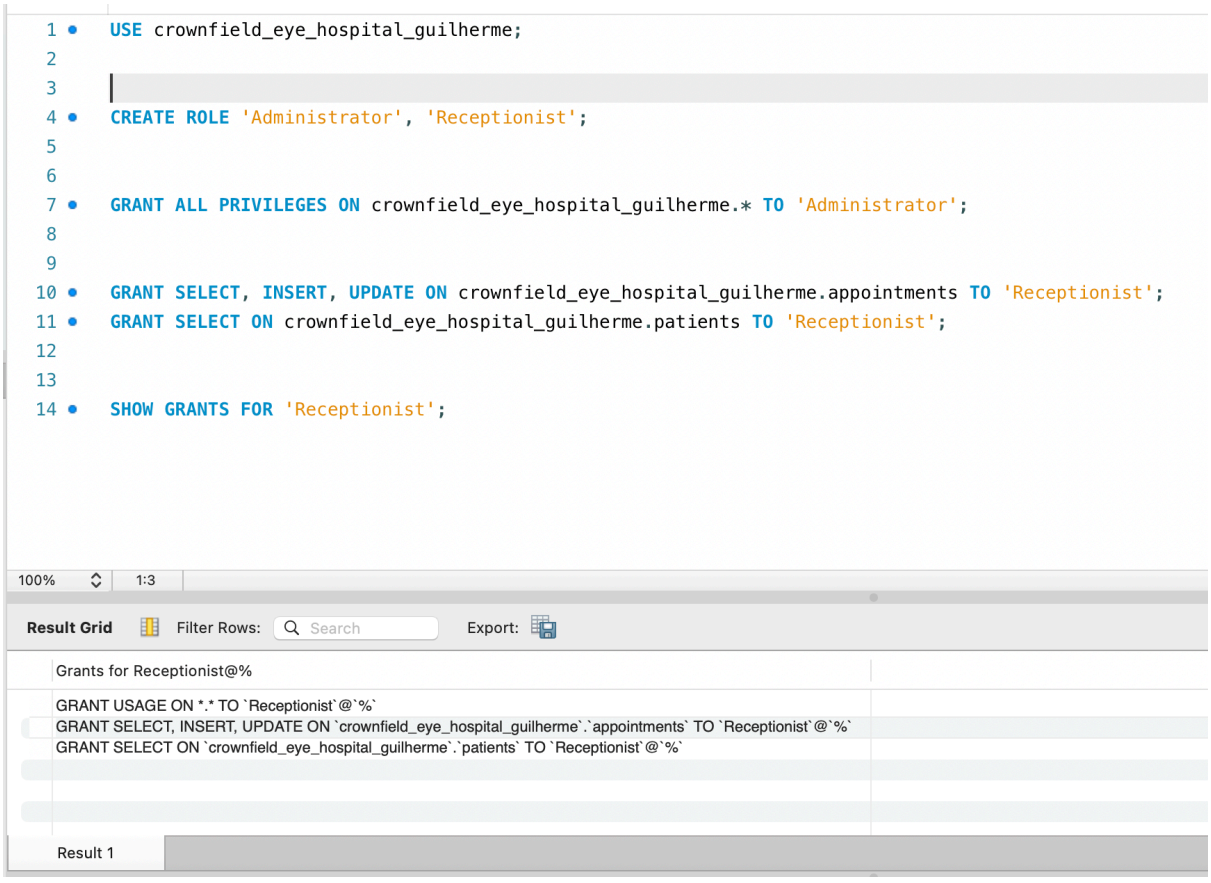
100% 59:16

Action Output

	Time	Action	Response
42	22:37:47	USE crownfield_eye_hospital_guilherme	0 row(s) affected
43	22:37:47	CREATE PROCEDURE GetPatientHistory(IN p_id VARCHAR(10)) BEGIN SELECT a.appointment_date AS 'D...	0 row(s) affected
44	22:37:47	CALL GetPatientHistory('P001')	1 row(s) returned
45	22:39:13	USE crownfield_eye_hospital_guilherme	0 row(s) affected
46	22:39:13	CREATE TRIGGER trg_prevent_past_appointments BEFORE INSERT ON appointments FOR EACH ROW BEGIN I...	0 row(s) affected
47	22:39:13	INSERT INTO appointments (patient_id, clinician_id, clinic_id, appointment_date, appointment_time, appointmen...	Error Code: 1644. Error: Cannot book an appointment in the past.

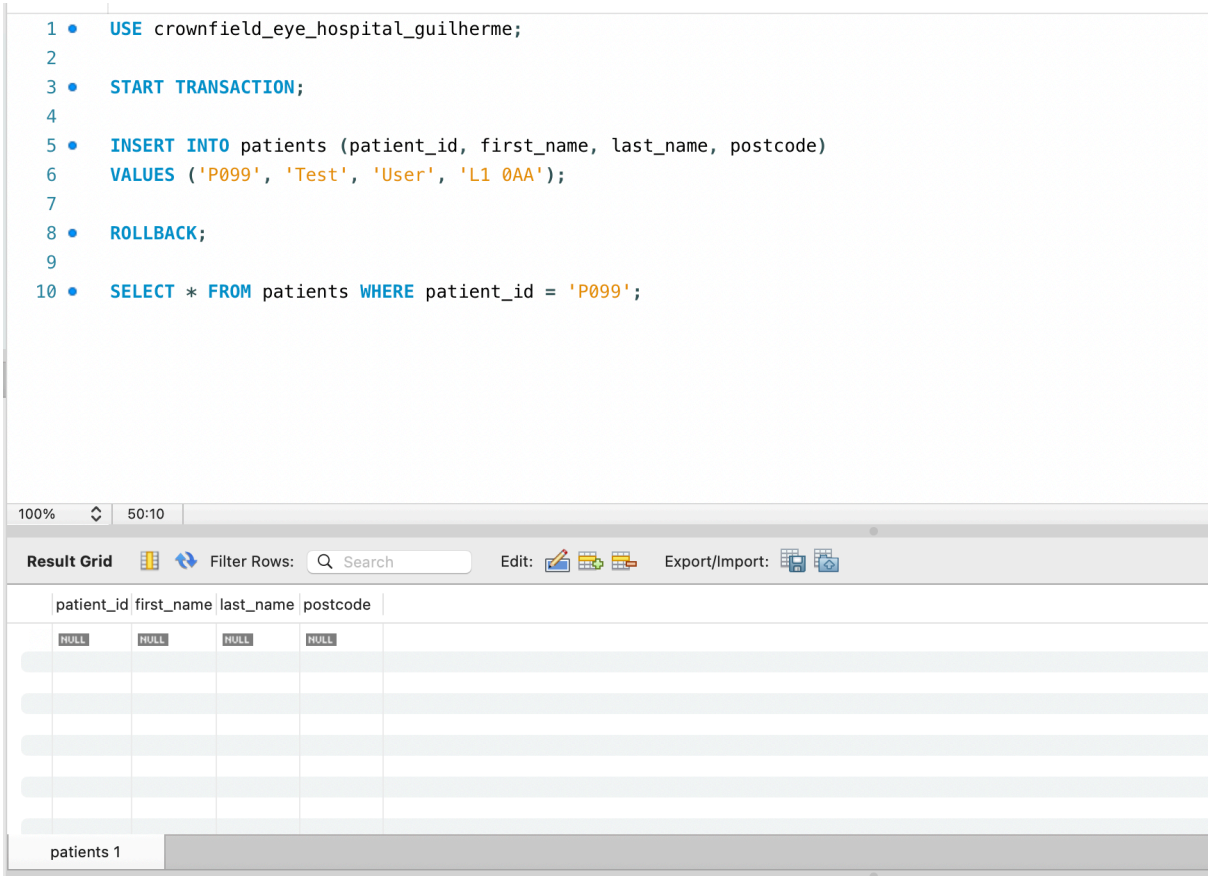
5. Security and transactions (Task D)

5.1 Role-Based Access Control (RBAC): The management of a healthcare database requires strict adherence to security protocols. The model was designed following the principle of least privilege. I created specific roles: the 'Administrator' user has full permissions (ALL PRIVILEGES), whilst the 'Receptionist' role was granted SELECT, INSERT, and UPDATE permissions on the appointments table alongside read-only SELECT access on patients, strictly withholding destructive privileges such as DELETE.



5.2 Data Protection and SQL Injection Defence: For data protection (Task D2), I highlighted the importance of hash algorithms (such as SHA-256) to protect sensitive information, ensuring that even DBAs cannot read confidential data directly. Furthermore, I documented the use of *Prepared Statements* (parameterised queries) in database interactions as the primary technical defence against *SQL Injection* attacks, preventing malicious inputs from manipulating the query structure.

5.3 Transaction Control and ACID Compliance: ACID transaction compliance was demonstrated using the START TRANSACTION and ROLLBACK commands. To prove engine-level rollback capabilities, a test patient record ('P099') was inserted within an active transaction and immediately rolled back. A subsequent SELECT query returned an empty result, verifying that uncommitted transactions leave no orphaned records or partial state changes in persistent storage.



6. Evaluation and reflection

6.1 System Evaluation and Production Limitations: The developed relational prototype fully meets the needs described in the *Crownfield Eye Hospital* scenario. The use of the third normal form eliminated the flaws identified in the original Excel-based records, reducing storage waste and preventing data obsolescence. The security tools and transaction controls proved that the system could operate robustly in a critical medical environment.

Despite the success of the prototype, there are limitations to be overcome prior to a production deployment. The first is the absence of an alteration history (audit tables). In a real healthcare scenario, tracking who altered a diagnosis and when is legally required. The second structural limitation is the absence of an automated *Disaster Recovery* plan. Although the system protects data against accidental deletions, for future iterations, the implementation of full and differential backup routines, stored encrypted on off-site servers, is strictly necessary. This measure would ensure that, in scenarios of catastrophic hardware failure or ransomware attacks, the ophthalmological history could be fully restored, ensuring the uninterrupted continuity of care.

6.2 Personal Reflection and Learning Outcomes: Working on this project deeply consolidated my technical capability. The process solidified my understanding that good database design occurs even before opening MySQL (LO1). Furthermore, practising advanced SQL, including procedures and triggers, gave me the confidence to manipulate and protect data efficiently (LO2 and LO3). The experience of debugging foreign key constraints was challenging, but crucial to understanding the importance of sequential planning of data dependencies.

7. References

- Connolly, T. and Begg, C. (2014) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th edn. Pearson.

- Coronel, C. and Morris, S. (2018) *Database Systems: Design, Implementation, & Management*. 13th edn. Cengage Learning.

- MySQL (2026) *MySQL 8.0 Reference Manual*. Available at: https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: 8 November 2026).